pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Deployment and Your Accounts

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: deployment and your accounts

Today

1. What deployment is, and where each piece runs
2. The free route, as each platform reads today
3. Your accounts: GitHub, and Hugging Face today
4. What origin is
5. A remote you own: push, clone, fetch, pull
6. Before anything goes public: public_scan
7. The first push to GitHub, masked
8. Masking a log before you share it

Every cloud step, two ways

On your own account

the PowerShell you run

A recorded transcript

the course's run, masked

No notebook cell reaches a platform.

Today has no server: the map, the accounts, a remote you own, and the scan.

The rules for deployed code

Checked first	nothing goes public until public_scan is empty
Synthetic bonds	a real company only as the author of a document
Described, never sold	a price or DV01 per 100 of face, with its as-of date
Firm policy first	free tiers are personal accounts for learning

On a platform's figures

Allowed names

included in Hobby as read

the free tier as read

the platform line

Never called

guaranteed

unlimited

free forever

How deployment works

The problem	Code that works on your laptop must work for other people and machines
The method	Four places, each checking what the last cannot: laptop, repository, test machine, server
The test machine	Fresh Linux for every change: installs what your files declare, runs the tests, is thrown away
You control	What goes into the repository and what your files declare
What comes out	A public address and a record of every change that reached it

What deployment is

Laptop

write and test

→

Repository

others read it

→

Test machine

nobody owns it

→

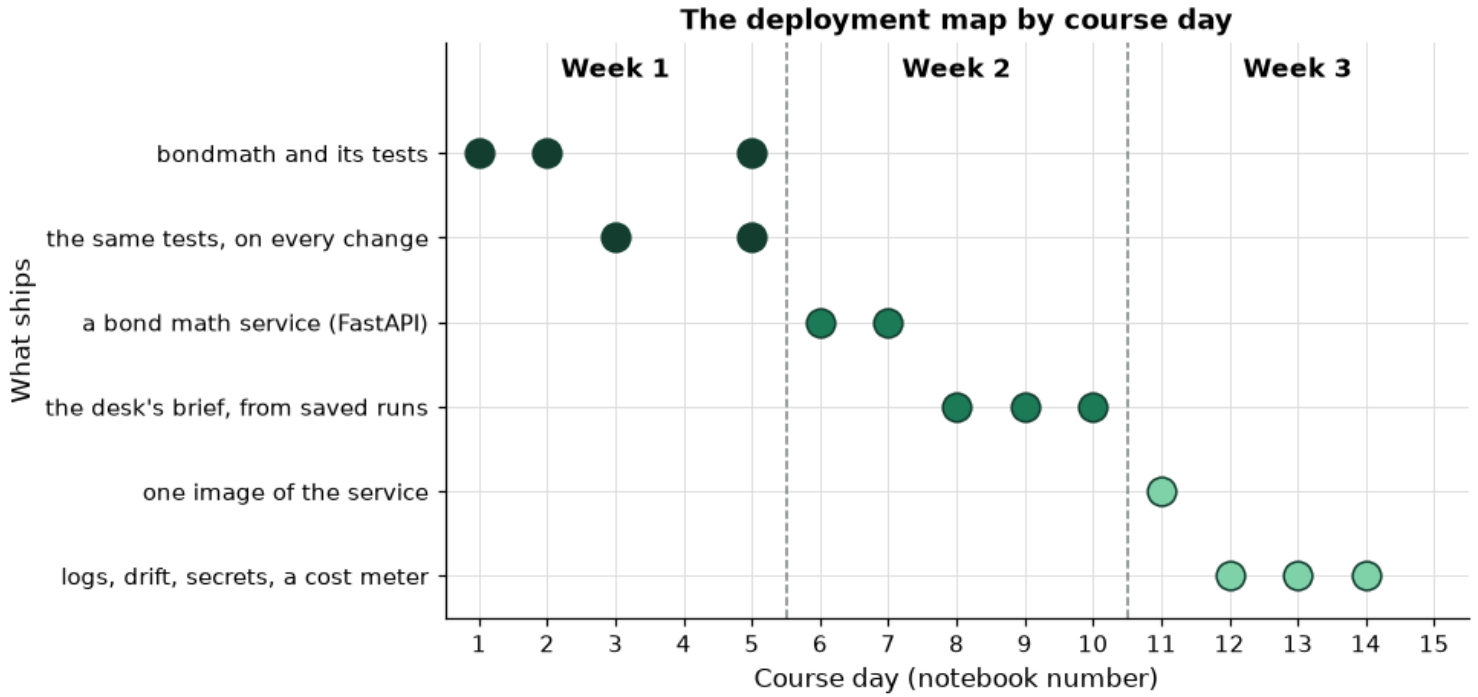
Server

a URL for anyone

Week 1	bondmath and its tests: a public repo; tests on Linux
Week 2	a bond service at a public URL; the desk's brief from saved runs
Week 3	one image in Docker; logs, drift, secrets, a cost meter

Every step after the first puts your code on a machine you do not sit at.

The map by course day



How to read it

Row

one step of the course

Dot

a course day that teaches it

Colour

the week it first ships

Week 1 stays on GitHub. Week 2 goes to Vercel and Hugging Face. Week 3 adds Docker and the upkeep.

The free route, as read today

GitHub

public repos; Actions on public repos

Vercel Hobby

1,000,000 invocations, 4 CPU hours

Hugging Face

2 ZeroGPU Spaces after 30 days

Docker Personal

$0, for individual developers

GitHub Actions: free on public repositories (docs.github.com/en/billing/concepts/product-billing/github-actions, read 2026-10-12).

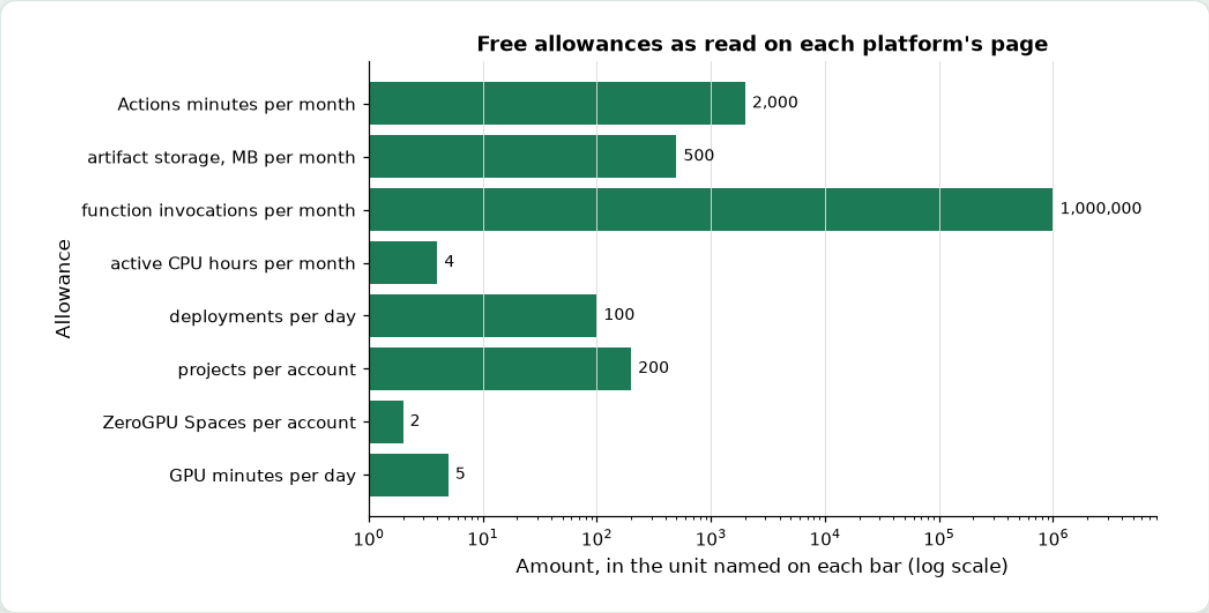
Vercel Hobby: 1,000,000 invocations and 4 CPU hours a month included (vercel.com/docs/plans/hobby, read 2026-10-12).

Hugging Face: 2 free ZeroGPU Spaces after 30 days, PRO $9 a month (huggingface.co/docs/hub/spaces-zerogpu and huggingface.co/pricing, read 2026-10-12).

Docker Personal: $0 (docker.com/pricing, read 2026-10-12).

The paid route is named where it fits, and never required: no lesson asks for a card.

The free allowances, drawn



Platform	Allowance	Per	Amount
GitHub Actions	Actions minutes	month	2,000
GitHub Actions	artifact storage, MB	month	500
Vercel Hobby	function invocations	month	1,000,000
Vercel Hobby	active CPU hours	month	4
Vercel Hobby	deployments	day	100
Vercel Hobby	projects	account	200
Hugging Face	ZeroGPU Spaces	account	2
Hugging Face	GPU minutes	day	5

GitHub Actions: free on public repositories (docs.github.com/en/billing/concepts/product-billing/github-actions, read 2026-10-12).

Vercel Hobby: 1,000,000 invocations and 4 CPU hours a month included (vercel.com/docs/plans/hobby, read 2026-10-12).

Hugging Face: 2 free ZeroGPU Spaces after 30 days, PRO $9 a month (huggingface.co/docs/hub/spaces-zerogpu and huggingface.co/pricing, read 2026-10-12).

Docker Personal: $0 (docker.com/pricing, read 2026-10-12).

Your accounts

GitHub

github.com/signup; verify your email

Settings, Emails:

Keep my email addresses private

commits then carry your noreply address

Hugging Face, today

huggingface.co/join; confirm your email

Settings, Access Tokens, New token

role: Write

`my_bondmath\.env:  HF_TOKEN=`

2026-10-12

the account is created

2026-10-21

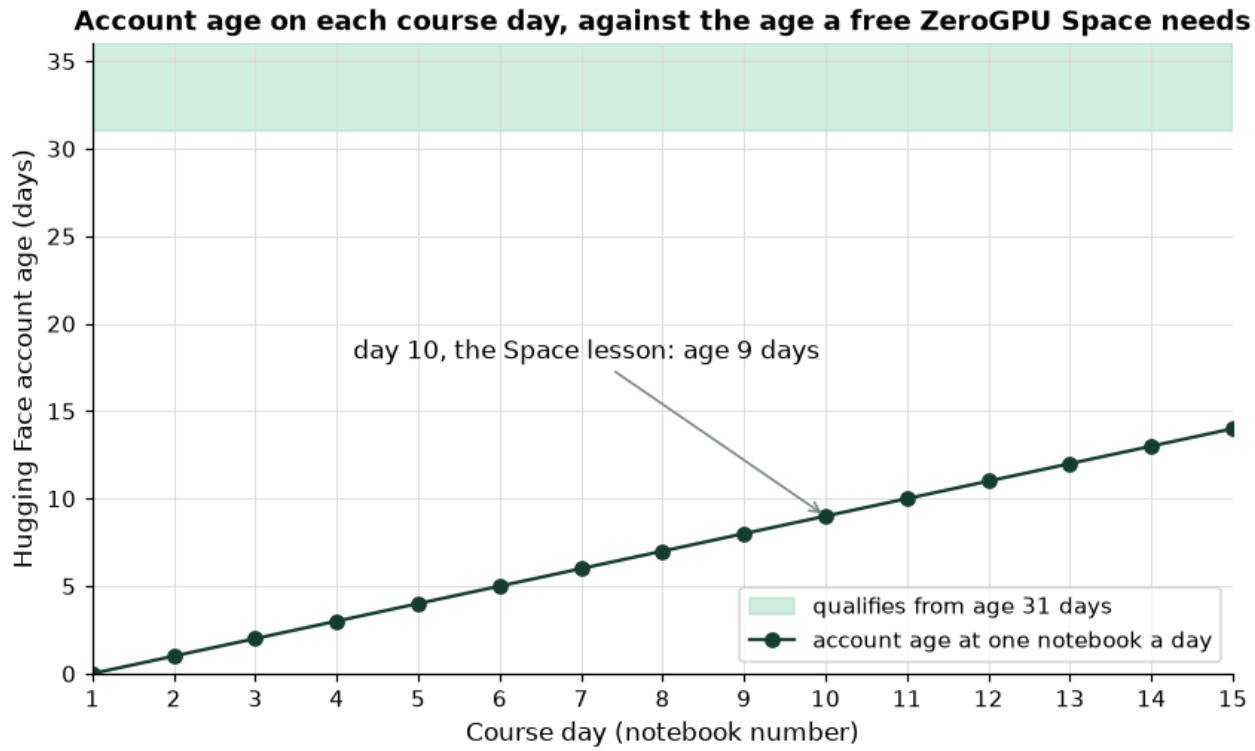
day 10 at one notebook a day

2026-11-12

qualifies: older than 30 days

```
HF token loaded: False.
```

The thirty-day rule, drawn



The Space step waits in its own cell. Nothing else in the course waits.

31 days

old enough for a free ZeroGPU Space

9 days

the account's age on the Space lesson

22 days

short of the rule on day 10

What origin is

What the platform printed, masked

```
Recorded 2026-10-12 on the course's own account; masked
> git remote -v
origin  https://github.com/student/python-fixed-income.git (fetch)
origin  https://github.com/student/python-fixed-income.git (push)
```

A remote: a name for another copy of the same repository.

origin: the one you cloned from. You can read from it; it is not yours to push to.

How a remote works

The problem	One history has to live in more than one place: yours, a server's, a colleague's
The remote	A name for another copy; a bare repository is that history with no working files
The verbs	Push sends your commits; fetch fills origin/main and leaves main; pull --ff-only moves main
You control	The remote's address and which branch each side tracks
What comes out	Copies of one history; git status -sb says ahead or behind

A remote you own

Work folder

your commits

→

remote.git

bare: history only

→

colleague

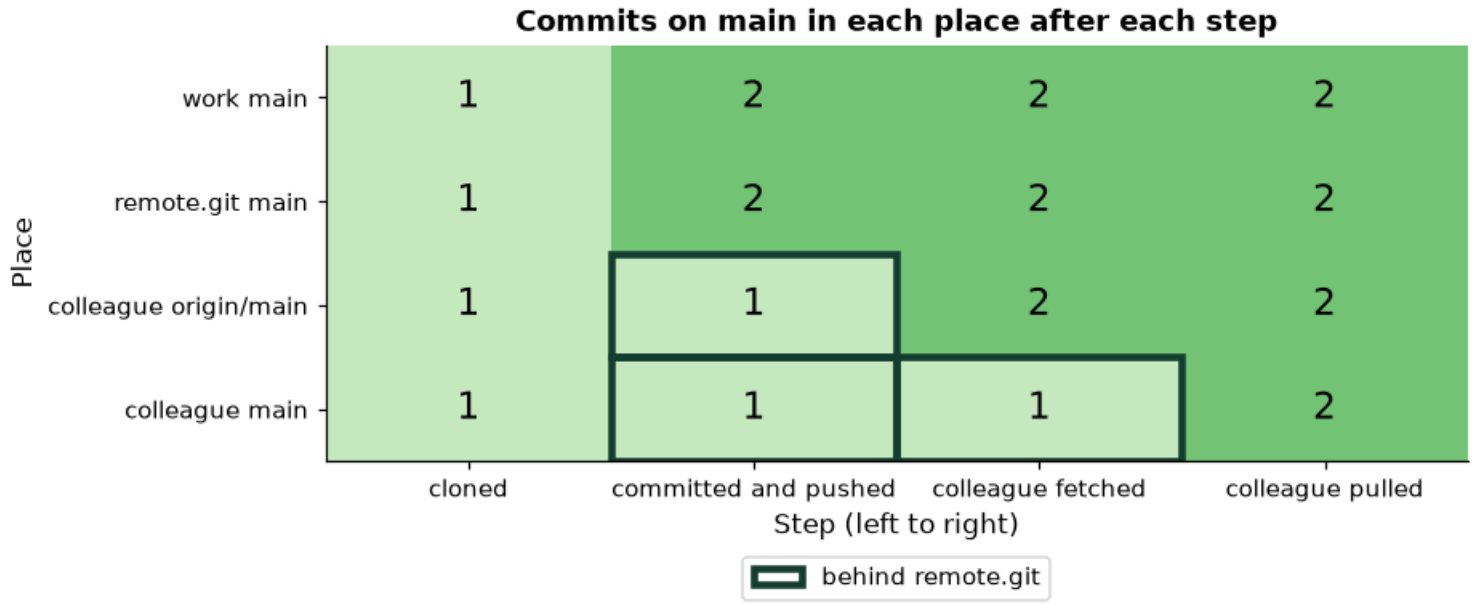
a clone

```
branch 'main' set up to track 'origin/main'.
To remote.git
 * [new branch]      main -> main
```

```
## main...origin/main [behind 1]
arrived in origin/main, not yet in main:
ab386b5 Ignore the colleague's clone
```

`push`	sends your commits to the remote
`fetch`	copies them to origin/main; main stays
`pull --ff-only`	moves main forward, or refuses

Where the newest commit is



How to read it

Column

one moment in time

Row

one place

Outline

a copy that is behind

Nothing reaches a copy until it asks. Fetch moves origin/main; pull moves main.

How public_scan works

The problem	A public repo keeps every earlier version of every file: check before the first commit
The method	Walk every file at any depth, skipping .git, and ask four questions of each
The questions	.env name? a key prefix on a line? an email but the course's? over the size limit?
You control	The folder and max_mb
What comes out	One row per problem, never the secret; an empty table is safe by these four rules

Before anything goes public

env_file

.env at any depth

key_like

sk-, hf_, ghp_, ...

email

any but the course's

large_file

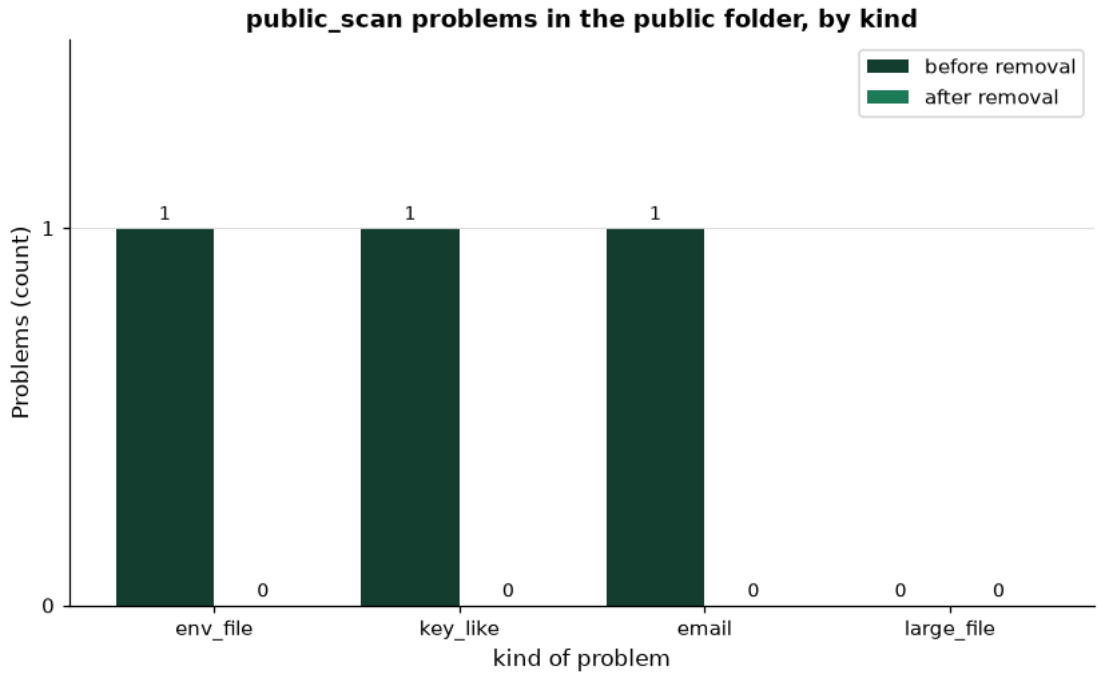
over max_mb

```
file  line     kind                                    detail
     .env     0 env_file        an environment file; never published
     .env     1 key_like sk-or- pattern, starts 'sk-', 18 characters
notes.txt     1    email     an address, starts 'des', 28 characters
problems: 3; safe to publish: no
```

problems: 0; safe to publish: yes; the tests in the folder alone: 40 passed

.gitignore protects a commit; the scan protects the folder. It never prints what it found.

The scan, before and after



Kind	Before	After
env_file	1	0
key_like	1	0
email	1	0
large_file	0	0

problems: 0; safe to publish: yes

A count of zero is the goal. A kind with no bar is a kind the scan looked for and did not find.

The first push to GitHub

```
git init -b main
git config user.name "<your name>"
git config user.email "<noreply>"
git add .
git commit -m "..."
git remote add origin <your repo>
git push -u origin main
```

What the platform printed, masked

Recorded 2026-10-12, the course's account, masked

```
> git push -u origin main
branch 'main' set up to track 'origin/main'.
To https://github.com/student/pfi-deploy-demo.git
 * [new branch]      main -> main
```

The same lines as the push to remote.git. Git Credential Manager signs you in: no token typed.

How masking works

The problem	A log names the account that ran it; sharing the log shares the name
The method	Names longest first, then emails, IPv4 addresses, platform hosts, and ids
Longest first	A team slug that holds a user name is replaced whole
You control	The list of names: the one input the function cannot guess
What comes out	The same text and shape with none of those strings; read it before you share it

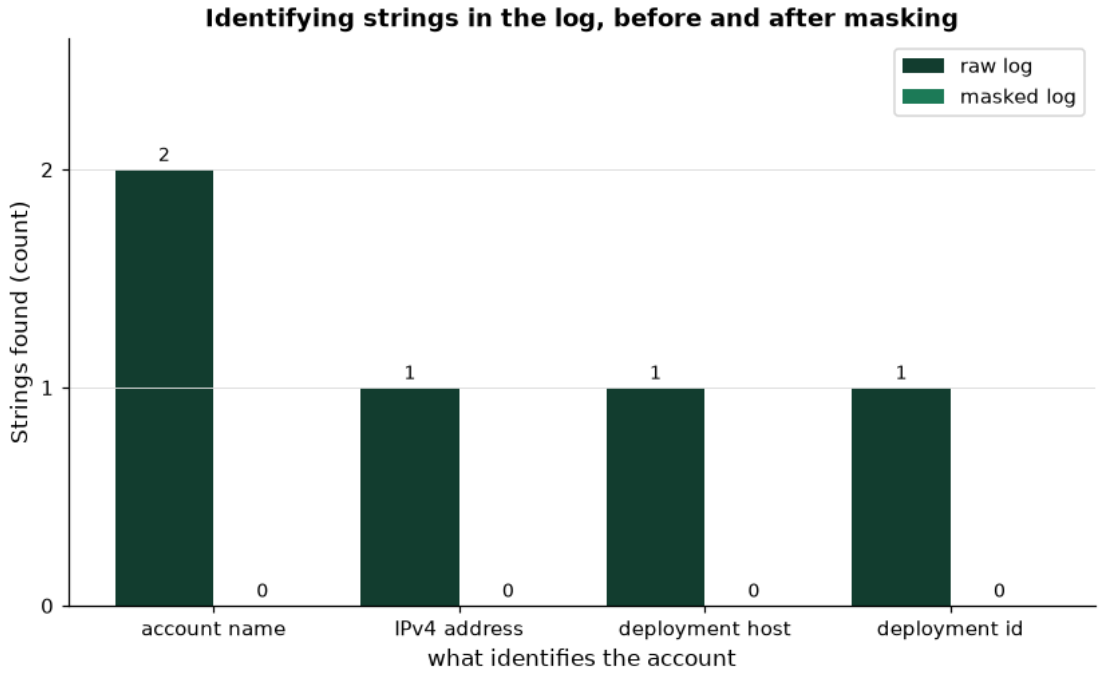
Masking a log before you share it

```
To https://github.com/student/pfi-bondmath.git
Connected from <ip>
Production: https://student.vercel.app (<id>)
```

account name	`student`
IPv4 address	`<ip>`
*.vercel.app, *.hf.space	`student`
Vercel deployment and request ids	`<id>`

Longer names go first, so a team slug goes whole. Read the result before you share it.

What the mask took out



Kind of string	Raw	Masked
account name	2	0
IPv4 address	1	0
deployment host	1	0
deployment id	1	0

These patterns find what this log holds. They are not a second mask.

Excel and Git

Excel side by side: none today

Excel has no remote, no clone, no push, and no scan of a folder. Week 3 brings it back.

```
deploykit day 1: run_git, public_scan, mask_transcript, show_transcript, 7 tests
Ignore the colleague's clone
Start deploykit beside bondmath and its tests: run_git, 1 test
```

```
.gitignore:3:.env   .env
```

Three commits: on the work folder, on remote.git, and in the colleague's clone. 7 tests.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: two globs at the top of the folder: deploy/config/.env.production with a key scans clean.

Code that runs is not the same as code that is right.

What you can do now

- Say what deployment is: four places
- Read the free route, with the page and the date
- Open GitHub and Hugging Face; the token in .env
- A remote you own: push, clone, fetch, pull
- public_scan before anything goes public; mask logs

Week 1, Day 2 Branches and pull requests